SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Scenario-Based : 20%

Total Marks: 50

Annexure A

Code of Conduct:

I _____HARI CHARAN P(192511107)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
HARI CHARAN P(192511107)

Annexure A

1. Secure Password Storage in Web Applications
A web application stores user passwords and needs to protect them against database breaches.

A. Explain why storing plaintext passwords is insecure.
B. Suggest a secure method for storing passwords and justify your choice.

---

2. Digital Signature for Document Authentication
A company wants to ensure that official documents sent electronically are authentic and unaltered.

A. Explain how digital signatures can be used in this scenario.
B. Justify how this method ensures both integrity and non-repudiation.

---

3. Preventing Replay Attacks in Online Transactions
An online payment system is vulnerable to replay attacks where attackers resend previously captured transactions.

A. Explain how replay attacks occur.
B. Suggest a cryptographic technique to prevent such attacks and justify your answer.

---

4. Secure Communication over Public Wi-Fi
A user accesses sensitive information using a public Wi-Fi network without encryption.

A. Identify the risks involved in this scenario.
B. Recommend a secure communication method and justify how it protects user data.

---

5. Data Integrity in Cloud Storage
An organization stores critical data in a cloud server and wants to ensure that the data is not altered.

A. Explain how hashing can be used to verify data integrity.
B. Suggest an additional mechanism to enhance trust and justify your answer.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

ANSWERS

1. Secure Password Storage in Web Applications

A. Why storing plaintext passwords is insecure

If passwords are stored as readable text, anyone who gets access to the database can immediately read every user's credentials. This can happen through SQL injection, a stolen backup, a misconfigured server or a malicious insider. The consequences are serious:

- **Direct account takeover:** an attacker logs in as any user without having to crack anything.
- **Credential reuse:** many users reuse the same password on other sites, so one breach exposes their email, banking and social accounts too.
- **Insider misuse:** administrators and developers with database access can see passwords they should never know.
- **No protection after a breach:** the damage is total and irreversible; the only remedy is forcing every user to reset their password.

B. Secure method: salted, slow password hashing

Passwords should never be stored; only a salted hash produced by a slow, purpose-built password hashing function such as bcrypt, scrypt, or Argon2 (PBKDF2 is also acceptable). The steps are:

- **Step 1:** when the user registers, generate a unique random salt (at least 16 bytes) for that user.
- **Step 2:** compute hash = KDF(password + salt, cost factor) and store only the salt, the cost parameters and the hash.
- **Step 3:** at login, hash the entered password with the stored salt and compare it with the stored hash using a constant-time comparison.

Justification:

- **One-way:** a hash cannot be reversed, so a stolen database does not reveal the passwords.
- **Salting:** a unique salt per user defeats rainbow tables and ensures that two users with the same password get different hashes, so an attacker must attack each account separately.
- **Slow and tunable:** bcrypt/Argon2 have an adjustable work factor (and Argon2 is memory-hard), which makes brute-force and GPU attacks very expensive, and the cost can be raised as hardware improves.
- **Why not plain SHA-256 or MD5:** fast general-purpose hashes allow billions of guesses per second, so they are unsuitable for passwords.

Optionally, a secret "pepper" held outside the database, enforcing strong password policies, and multi-factor authentication add further protection.

Additional Theory for Question 1: Password Security

1. Cryptographic hash function: required properties

A hash function H maps an input of any length to a fixed-length output (digest). For password protection the following properties matter:

- **Deterministic:** the same input always gives the same digest, which is what makes login verification possible.
- **Pre-image resistance (one-way):** given a digest h, it is computationally infeasible to find any input m such that H(m) = h.
- **Second pre-image resistance:** given m1, it is infeasible to find a different m2 with H(m1) = H(m2).
- **Collision resistance:** it is infeasible to find any two different inputs that produce the same digest.
- **Avalanche effect:** changing a single bit of the input changes about half the bits of the output.

2. Common attacks on stored passwords

- **Brute-force attack:** trying every possible password until the hash matches; its cost grows exponentially with password length and complexity.
- **Dictionary attack:** trying a list of common words and leaked passwords, often with rules such as adding digits or substituting letters.
- **Rainbow-table attack:** using precomputed hash chains to reverse unsalted hashes quickly; salting makes these tables useless.
- **Credential stuffing:** using username/password pairs leaked from one site to log in on others.
- **Offline cracking with GPUs/ASICs:** after a database theft the attacker can guess at full hardware speed, which is why the hash must be deliberately slow.

3. Salt, pepper and key stretching

- **Salt:** a unique random value stored alongside the hash, not secret. It guarantees unique hashes and forces per-user attacks.
- **Pepper:** a secret value kept outside the database (for example in an HSM or secrets vault) and mixed into the hash, so a database-only leak is not enough to crack passwords.
- **Key stretching:** repeating the hash many times (PBKDF2 iterations) or using a memory-hard design (scrypt, Argon2) to make each guess expensive for the attacker but only slightly slower for a legitimate login.

4. Comparison of password storage methods

Method	Security level	Remarks
Plaintext	None	Passwords are readable by anyone with database access. Never acceptable.

Encryption (reversible)	Weak	The key must be stored somewhere; if it leaks, all passwords are exposed. Reversibility is unnecessary for authentication.
MD5 / SHA-1 / SHA-256 (unsalted)	Poor	Very fast, so billions of guesses per second; vulnerable to rainbow tables.
Salted SHA-256	Moderate	Defeats rainbow tables but is still too fast against GPU brute force.
PBKDF2	Good	Salted, configurable iteration count; widely standardised (NIST, FIPS compliant).
bcrypt	Strong	Salted, adaptive cost factor, based on the Blowfish cipher; resistant to GPU attacks.
scrypt / Argon2id	Very strong	Memory-hard, which limits parallel attacks on GPUs and ASICs; Argon2 won the Password Hashing Competition (2015) and is the current recommendation.

5. Best practices for secure password storage

- Enforce a minimum length (for example 12 characters or more) and check new passwords against lists of breached passwords, instead of relying only on complexity rules.
- Always transmit passwords over TLS (HTTPS) and never log or email them.
- Rate-limit login attempts and add account lockout or CAPTCHA to slow online guessing.
- Re-hash automatically at login when the cost parameters are increased or the algorithm is upgraded.
- Offer multi-factor authentication so that a stolen password alone is not enough.

Conclusion: because users cannot be relied upon to choose strong or unique passwords, the application must assume that its database may eventually be stolen. Storing only salted hashes made with a slow, memory-hard algorithm limits the damage of such a breach and keeps user credentials safe.

2. Digital Signature for Document Authentication

A. How digital signatures can be used

A digital signature uses public-key cryptography (for example RSA or ECDSA together with SHA-256) to bind a document to its sender. The process is:

- **Signing (sender):** the company computes a hash (message digest) of the official document and encrypts that digest with its private key. The result is the digital signature, which is sent with the document, normally along with the company's digital certificate issued by a trusted Certificate Authority.

• **Verification (receiver):** the receiver decrypts the signature using the company's public key to recover the original digest, independently hashes the received document, and compares the two values.

• **Outcome:** if the two digests match, the document is authentic and unaltered; if they differ, the document was modified or was not signed by the claimed sender, and it is rejected.

B. How it ensures integrity and non-repudiation

• **Integrity:** the signature depends on the hash of the exact document content. Changing even a single character produces a completely different hash, so verification fails. An attacker cannot create a valid signature for a modified document without the private key.

• **Authentication:** only the holder of the private key can produce a signature that the matching public key verifies, and the CA-issued certificate links that key to the company's identity.

• **Non-repudiation:** since the private key is known only to the signer, the company cannot later deny having signed the document. Any third party (an auditor or a court) can verify the signature with the public key, and timestamps and certificates provide additional proof of when and by whom it was signed. This is something a shared-secret mechanism such as a MAC cannot provide, because both parties hold the same key.

Hence digital signatures give a document that is authentic, tamper-evident and legally binding.

Additional Theory for Question 2: Digital Signatures

1. Definition and security services

A digital signature is a cryptographic value calculated from the data and the signer's private key, which allows anyone with the corresponding public key to verify the origin and integrity of that data. It is the electronic counterpart of a handwritten signature, but it is tied to the content of each document and therefore cannot be copied from one document to another. It provides three services: authentication, integrity and non-repudiation. Confidentiality is not provided; if needed, the document must also be encrypted.

2. Mathematical working (RSA signature)

• **Key generation:** choose primes p and q, compute $n = p \times q$, choose a public exponent e and compute the private exponent d such that $e \times d \equiv 1 \pmod{\varphi(n)}$. The public key is (e, n) and the private key is d.

• **Signing:** compute the digest $h = H(M)$ and then the signature $S = h^d \bmod n$.

• **Verification:** compute $h' = S^e \bmod n$ and the digest H(M') of the received message; the signature is valid if $h' = H(M')$.

Because only the owner of d can produce S, a valid result proves that the message came from the key owner. Real systems use padding schemes such as RSA-PSS rather than raw RSA.

3. Digital signature algorithms

Algorithm	Based on	Remarks
RSA (RSA-PSS)	Integer factorisation	Most widely deployed; larger keys needed (2048 bits or more).
DSA / DSS	Discrete logarithm	NIST Digital Signature Standard; signing needs a good random number per signature.
ECDSA	Elliptic-curve discrete logarithm	Smaller keys and faster operations at the same security level; used in TLS, Bitcoin and many identity systems.
EdDSA (Ed25519)	Twisted Edwards curves	Fast, deterministic and resistant to many implementation errors.

4. Role of PKI and digital certificates

The public key must be trusted to belong to the claimed sender. A Public Key Infrastructure (PKI) solves this: a trusted Certificate Authority (CA) issues an X.509 digital certificate that binds the company's identity to its public key and is itself signed by the CA. The receiver validates the certificate chain up to a trusted root, checks the validity period, and checks whether the certificate has been revoked using a Certificate Revocation List (CRL) or the Online Certificate Status Protocol (OCSP). A trusted timestamp can also be added so that the signature remains verifiable evidence even after the certificate expires.

5. Digital signature compared with MAC

Feature	Digital signature	MAC (e.g. HMAC)
Key type	Asymmetric (private key signs, public key verifies)	Symmetric (shared secret key)
Integrity	Yes	Yes
Authentication	Yes, identifies the individual signer	Only shows the sender is one of the key holders
Non-repudiation	Yes	No, as both parties can create the tag
Speed	Slower	Faster

6. Possible attacks and threats

- **Forgery:** creating a valid signature without the private key; prevented by strong algorithms, adequate key sizes and secure hash functions (SHA-2/SHA-3).
- **Collision attacks on weak hashes:** MD5 and SHA-1 collisions have allowed forged documents, so they must not be used for signing.
- **Private key compromise:** if the key is stolen, attacker signatures look genuine; keys should be kept in smart cards or Hardware Security Modules and certificates revoked immediately on compromise.
- **Replay of signed documents:** a valid old document can be resent; timestamps, serial numbers or nonces inside the document help prevent this.

7. Applications and legal status

Digital signatures are used in e-governance and e-filing, signed PDF contracts and invoices, software and firmware updates, code signing, secure email (S/MIME), TLS/SSL certificates and blockchain transactions. In India, digital signatures issued by licensed Certifying Authorities are legally recognised under the Information Technology Act, 2000, and similar laws exist elsewhere (the eIDAS regulation in the EU and the ESIGN Act in the US).

Conclusion: for the company's official documents, a hash-then-sign scheme using a certificate-backed key pair gives a verifiable guarantee of origin, protects against tampering and prevents the sender from denying the document later, which is why it is the standard solution for authenticating electronic documents.